

Prompt:

8/17/2026, 10:11:54 PM

lanjut Tahap Y

Response:

8/17/2026, 10:11:54 PM

Remembering

Reconstructed and checked the A-X framework, mappings, and factor design

Worked for 5s

Tahap Y — Unified Mathematical Formulation

Ya. Setelah Tahap X, sekarang kita sampai pada bagian yang menurut saya **paling penting secara matematis**: kita berhenti melihat sistem sebagai kumpulan modul terpisah, lalu menuliskannya sebagai **satu sistem keputusan stokastik yang utuh**.

Kerangka besarnya adalah:

$$F_t \rightarrow s_t \rightarrow (\hat{\mu}_t, \hat{\Sigma}_t) \rightarrow Q_t \rightarrow x_t^* \rightarrow w_t^* \rightarrow q_t^* \rightarrow O_t \rightarrow E_{t+1} \rightarrow R_{t+1} \rightarrow F_{t+1}$$

dengan feedback:

$$\text{Market} \rightarrow \text{Decision} \rightarrow \text{Execution} \rightarrow \text{Position} \rightarrow \text{Risk} \rightarrow \text{Re-optimization.}$$

Ini pada dasarnya menjadikan sistem kita sebagai **closed-loop stochastic control system**.

Y.1. Titik awal: information filtration

Kita mulai dari konsep yang sangat fundamental.

Pada waktu t , trader tidak mengetahui masa depan.

Definisikan:

$$F_t = \sigma(D_0, D_1, \dots, D_t)$$

sebagai sigma-algebra seluruh informasi yang tersedia sampai waktu t .

Informasi tersebut dapat terdiri atas:

$$D_t = \{P_t, V_t, OB_t, F_t, M_t, E_t\}.$$

Misalnya:

- P_t : harga,
- V_t : volume,
- OB_t : order book,
- F_t : market factors,
- M_t : market regime,
- E_t : execution/position state.

Maka seluruh keputusan kita harus memenuhi:

$$a_t \in F_t.$$

Ini adalah constraint fundamental:

Tidak boleh ada future information dalam keputusan trading.

Y.2. State sistem

Sekarang kita bentuk state vector.

Kita definisikan:

$$s_t = \begin{bmatrix} S_t^{market} \\ S_t^{factor} \\ S_t^{portfolio} \\ S_t^{risk} \\ S_t^{execution} \end{bmatrix}.$$

Lebih eksplisit:

$$s_t = [P_t, V_t, OB_t, F_t, R_t, w_t, q_t, c_t, \rho_t, \tau_t].$$

Dengan:

$$w_t$$



adalah portfolio weight,

$$q_t$$

adalah posisi/quantity,

$$c_t$$

adalah cash,

$$\rho_t$$

adalah risk state,

dan

$$\tau_t$$

adalah status order/position.

Y.3. Market factor vector

Karena kita ingin model berlaku baik untuk crypto maupun saham internasional, kita tidak boleh mendesain factor model khusus crypto.

Definisikan:

$$\mathbf{f}_t = \begin{bmatrix} f_t^{\text{market}} \\ f_t^{\text{macro}} \\ f_t^{\text{sector}} \\ f_t^{\text{crossasset}} \end{bmatrix}.$$

Untuk crypto, misalnya:

$$\mathbf{f}_t = [r_{BTC}, r_{ETH}, r_{TOTAL}, r_{DXY}, r_{Gold}, r_{VIX}, Funding, OI, \dots].$$

Untuk saham:

$$\mathbf{f}_t = [r_{SPX}, r_{NDX}, r_{Sector}, r_{DXY}, r_{Gold}, r_{US10Y}, r_{VIX}, \dots].$$

Dengan demikian asset universe berubah, tetapi mathematical engine tetap sama.

Y.4. Cross-asset dependency

Misalkan return asset i :

$$r_{i,t}.$$

Kita modelkan:

$$r_{i,t} = \alpha_i + \beta_i^\top f_t + \epsilon_{i,t}.$$

atau secara vector:

$$r_t = \alpha + Bf_t + \epsilon_t.$$

Ini sangat penting.

Kita tidak lagi mengatakan:

"Harga altcoin naik."

Kita mencoba memahami:

"Apakah pergerakan altcoin dapat dijelaskan oleh perubahan BTC, ETH, DXY, Gold, volatility, dan factor lainnya?"

Y.5. Expected return

Dari model tersebut:

$$E[r_t | F_{t-1}] = \alpha + Bf_t.$$

Kita definisikan:

$$\hat{\mu}_t = E[r_{t+1} | F_t].$$

Jadi:

$$\hat{\mu}_t = \alpha_t + \hat{B}_t f_t.$$

Ini menjadi input optimizer.

Y.6. Covariance

Kita juga membutuhkan:

$$\hat{\Sigma}_t = \text{Cov}(r_{t+1} | F_t).$$

Dari factor model:

$$r_t = Bf_t + \epsilon_t.$$

Jika:

$$\text{Cov}(f_t) = \Sigma_f$$

dan:

$$\text{Cov}(\epsilon_t) = D,$$

maka:

$$\Sigma = B\Sigma_f B^T + D.$$

Ini memberi kita covariance matrix yang lebih terstruktur daripada sekadar sample covariance.

Y.7. Mengapa factor model penting?

Karena misalnya kita memiliki:

$$N = 500$$

asset.

Sample covariance membutuhkan:

$$\frac{N(N-1)}{2}$$

elemen covariance.

Untuk $N = 500$:

$$124\,750$$

elemen.

Sedangkan factor model dengan:

$$K = 10$$

factor hanya membutuhkan struktur:

$$B \in \mathbb{R}^{500 \times 10}.$$

Ini jauh lebih manageable.

Y.8. Market regime

Namun:

$$\hat{\mu}_t$$

dan:

$$\hat{\Sigma}_t$$

tidak boleh dianggap stationary.

Kita definisikan latent regime:

$$z_t \in \{1, \dots, K\}.$$

Misalnya:

$$z_t = \begin{cases} 1 & \text{low volatility} \\ 2 & \text{bull} \\ 3 & \text{bear} \\ 4 & \text{high volatility.} \end{cases}$$

Y.9. Conditional distribution

Maka:

$$r_{t+1} \mid z_t = k \sim D_k(\mu_k, \Sigma_k).$$

Sehingga:

$$\hat{\mu}_t = \mu_{z_t}$$

dan:

$$\hat{\Sigma}_t = \Sigma_{z_t}.$$

Atau versi lebih fleksibel:

$$\hat{\mu}_t = f_\theta(s_t),$$

$$\hat{\Sigma}_t = g_\phi(s_t).$$

Y.10. Sekarang masuk ke portfolio

Misalkan:

$$w_t \in \mathbb{R}^N$$

adalah vector portfolio weight.

Portfolio return:

$$r_{p,t+1} = w_t^\top r_{t+1}.$$

Expected portfolio return:

$$\mathbb{E}[r_{p,t+1} | \mathcal{F}_t] = w_t^\top \hat{\mu}_t.$$

Variance:

$$\text{Var}(r_{p,t+1} | \mathcal{F}_t) = w_t^\top \hat{\Sigma}_t w_t.$$



Y.11. Risk-adjusted objective

Kita dapat memulai dari Markowitz:

$$J(w) = w^T \hat{\mu}_t - \lambda_t w^T \hat{\Sigma}_t w.$$

dengan:

$$\lambda_t > 0.$$

Namun untuk short-term trading, ini belum cukup.

Kita membutuhkan transaction cost.

Y.12. Transaction cost

Definisikan perubahan portfolio:

$$\Delta w_t = w_t - w_{t-1}.$$

Biaya:

$$C_t(\Delta w_t).$$

Model sederhana:

$$C_t = c^T |\Delta w_t|.$$

Sehingga objective:

$$J_t(w) = w^T \hat{\mu}_t - \lambda_t w^T \hat{\Sigma}_t w - c^T |w - w_{t-1}|.$$

Sekarang optimizer tidak hanya mencari return tinggi.

Ia juga mempertimbangkan:

"Apakah perubahan posisi ini layak dibandingkan biaya melakukan perubahan?"

Y.13. Market impact

Untuk trading lebih realistis:

$$C_t(\Delta w) = c^T |\Delta w| + \eta \|\Delta w\|_2^2.$$

Term pertama:

$$c^T |\Delta w|$$

merepresentasikan spread/fee.

Term kedua:

$$\eta \|\Delta w\|_2^2$$

merepresentasikan market impact.

Sehingga:

$$J_t(w) = w^T \mu_t - \lambda_t w^T \Sigma_t w - c^T |\Delta w| - \eta \|\Delta w\|^2.$$

Ini jauh lebih dekat dengan problem institutional portfolio rebalancing.

Y.14. Risk budget

Sekarang kita masukkan filosofi yang sebelumnya kita bangun:

Setiap trade harus memiliki bounded risk.

Misalkan portfolio capital:

$$C_t.$$

Risk budget:

$$R_t.$$

Jika kita ingin maksimum loss:

$$R_t = \$1$$



untuk sebuah trade, maka:

$$R_{trade} \leq R_t.$$

Untuk sistem general:

$$R_t = \alpha_t C_t.$$

Y.15. Position sizing dari stop-loss

Misalkan:

$$p_t$$

adalah entry price.

Stop loss:

$$s_t.$$

Risk per unit:

$$\delta_t = |p_t - s_t|.$$

Jika quantity:

$$q_t,$$

maka:

$$Risk_t = q_t \delta_t.$$

Maka constraint:

$$q_t \delta_t \leq R_t.$$

Sehingga:

$$q_t \leq \frac{R_t}{\delta_t}.$$

Ini adalah jembatan langsung antara:

Optimization

dan:

Execution.

Y.16. Expected return vs risk budget

Sekarang kita mempunyai dua layer:

Layer portfolio

$$w_t^* = \arg \max_w J_t(w).$$

Layer execution

$$q_t^* = \min(q_t^{\text{portfolio}}, \frac{R_t}{\delta_t}).$$

Jadi optimizer tidak boleh secara langsung menentukan quantity tanpa risk constraint.

Y.17. Entry decision

Sekarang kita masuk ke short-term trading.

Untuk setiap asset i , definisikan:

$$a_{i,t} \in \{-1, 0, +1\}.$$

dengan:

-1 = short,

0 = flat,

$+1$ = long.

Maka action vector:

$$a_t \in \{-1, 0, +1\}^N.$$

Y.18. Mengapa discrete action?

Karena order placement sebenarnya discrete:

Buy

atau:

Sell

atau:

No Trade.

Inilah yang membuat masalah ini cocok untuk:

QUBO

dan:

Ising.

Y.19. Binary representation

Untuk long-only:

$$x_i \in \{0, 1\}.$$

Untuk long/short kita membutuhkan dua binary variable:

$$x_i^+, \quad x_i^-.$$

dengan constraint:

$$x_i^+ + x_i^- \leq 1.$$

Kemudian:

$$a_i = x_i^+ - x_i^-.$$

Y.20. Portfolio selection QUBO

Misalkan:

$$x_i \in \{0, 1\}.$$

Objective:

$$J(x) = \mu^T x - \lambda x^T \Sigma x.$$

Untuk minimization:

$$H(x) = -\mu^T x + \lambda x^T \Sigma x.$$

Inilah QUBO fundamental kita.

Y.21. Cardinality constraint

Misalnya maksimum:

$$K$$

asset.

Constraint:

$$\sum_i x_i = K.$$

Masukkan penalty:

$$P \left(\sum_i x_i - K \right)^2.$$



Maka:

$$H(x) = -\mu^\top x + \lambda x^\top \Sigma x + P \left(\sum_i x_i - K \right)^2.$$

Y.22. Transaction-aware QUBO

Kita juga ingin memasukkan turnover.

Misalkan portfolio sebelumnya:

$$x_i^{old}.$$

Turnover:

$$|x_i - x_i^{old}|.$$

Maka:

$$H(x) = -\mu^\top x + \lambda x^\top \Sigma x + \kappa \sum_i |x_i - x_i^{old}|.$$

Karena:

$$x_i, x_i^{old} \in \{0, 1\},$$

kita memiliki:

$$|x_i - x_i^{old}| = x_i + x_i^{old} - 2x_i x_i^{old}.$$

Jadi seluruh objective tetap dapat ditulis sebagai quadratic polynomial.

Y.23. Sekarang faktor macro masuk ke QUBO secara tidak langsung

Ini sangat penting.

Kita **tidak perlu** membuat:

$$DXY$$

atau:

$$Gold$$

menjadi qubit tersendiri.

Mereka memengaruhi:

$$\hat{\mu}_t$$

dan:

$$\hat{\Sigma}_t.$$

Sehingga:

$$DXY_t \rightarrow \hat{\mu}_t \rightarrow H_t(x).$$

Contoh:

$$DXY \uparrow$$

dapat mengubah:

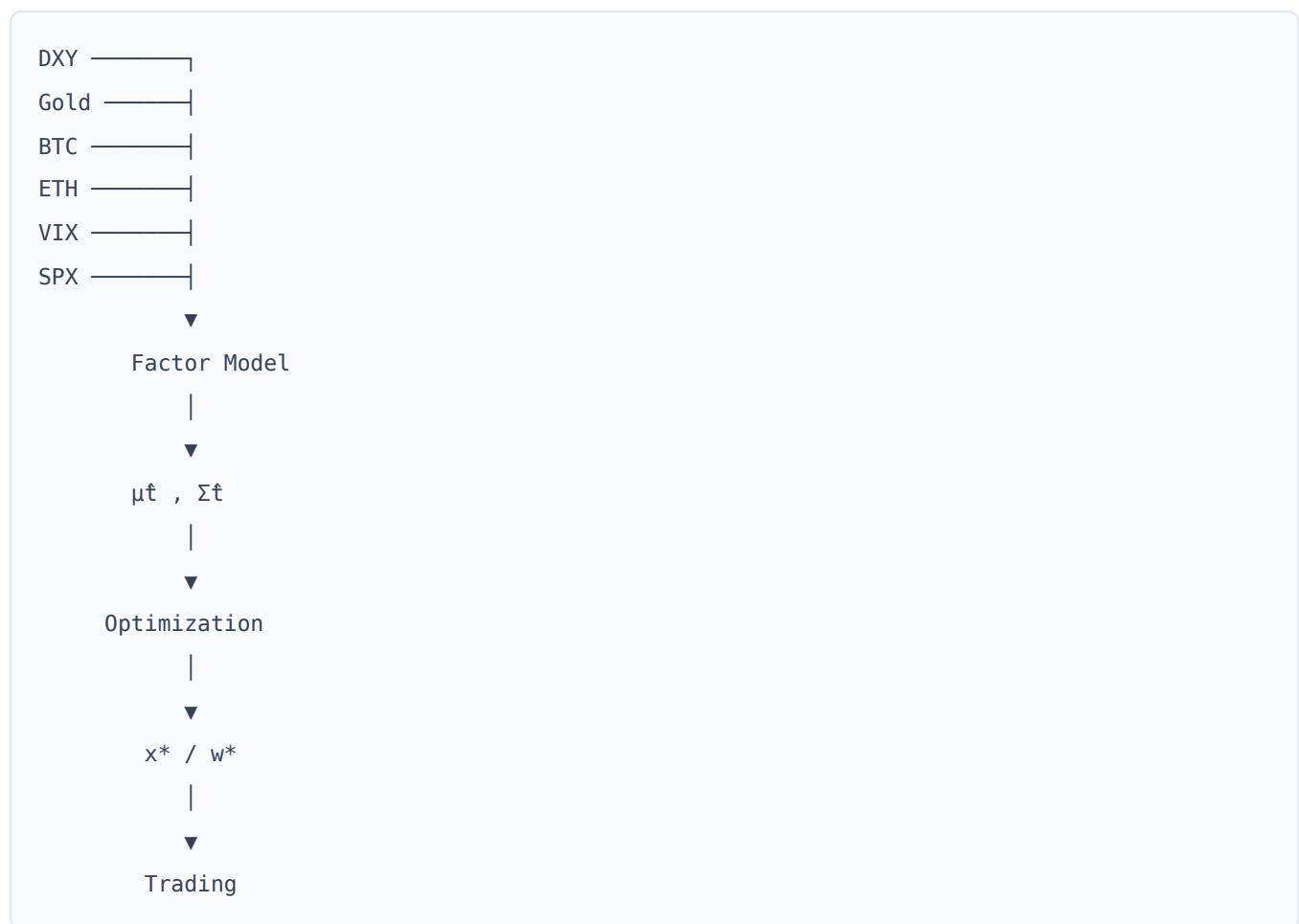
$$\hat{\mu}_{crypto}$$

menjadi lebih rendah.

Kemudian optimizer otomatis mengubah:

$$x^*.$$

Y.24. Ini adalah arsitektur yang kita inginkan



Jadi macro factor tidak langsung menentukan order.

Ia mempengaruhi **conditional expectation**.

Y.25. Regime-dependent risk aversion

Sekarang kita memanfaatkan regime.

Kita definisikan:

$$\lambda_t = \lambda(z_t).$$

Contoh:

$$\lambda_{bull} < \lambda_{normal} < \lambda_{crisis}.$$

Dalam crisis:

$$\lambda_t \uparrow .$$

Sehingga penalti risk meningkat:

$$\lambda_t w^T \Sigma w.$$

Akibatnya portfolio secara otomatis menjadi lebih defensif.

Y.26. Dynamic risk budget

Risk budget juga dapat bergantung pada regime:

$$R_t = R_{base} g(z_t, \sigma_t, \text{liquidity}_t).$$

Misalnya:

$$g < 1$$

ketika:

$$\sigma_t$$

tinggi.

Jadi:

$$Volatility \uparrow \Rightarrow PositionSize \downarrow .$$

Y.27. Entry signal

Kita sekarang membutuhkan threshold.

Misalkan expected excess return:

$$\alpha_{i,t} = \hat{\mu}_{i,t} - r_{benchmark,t}.$$

Trade hanya jika:

$$|\alpha_{i,t}| > \theta_i.$$

Tetapi threshold juga dapat mempertimbangkan uncertainty:

$$\boxed{\frac{|\hat{\mu}_{i,t}|}{\hat{\sigma}_{i,t}} > \theta.}$$

Ini lebih masuk akal daripada hanya melihat return prediction.

Y.28. Signal-to-risk ratio

Definisikan:

$$SR_{i,t}^{local} = \frac{\hat{\mu}_{i,t}}{\sqrt{\hat{\Sigma}_{ii,t}}}.$$

Kemudian:

$$a_{i,t} = \begin{cases} +1, & SR_{i,t} > \theta \\ 0, & |SR_{i,t}| \leq \theta \\ -1, & SR_{i,t} < -\theta. \end{cases}$$

Namun dalam sistem kita, ini hanya **candidate signal**.

Final decision masih berasal dari optimizer.

Y.29. Candidate set

Kita tidak perlu mengoptimalkan seluruh dunia.

Definisikan:

$$C_t = \{i : |\alpha_{i,t}| > \theta\}.$$

Kemudian optimizer bekerja pada:

$$N_t = |C_t|.$$

Ini sangat penting untuk scalability.

Y.30. Order generation

Setelah mendapatkan:

$$x_t^*,$$

kita menentukan:

$$entry_t, SL_t, TP_t, q_t.$$

Jadi:

$$\boxed{x_t^* \rightarrow TradePlan_t.}$$

Kita definisikan:

$$TradePlan_t = (direction, entry, SL, TP, quantity, expiry).$$

Y.31. Stop-loss mathematically

Untuk long:

$$SL_t = P_t - \delta_t.$$

Untuk short:

$$SL_t = P_t + \delta_t.$$

Dengan:

$$\delta_t = k_o \sigma_{i,t}$$

atau volatility-adjusted distance.

Misalnya:

$$\boxed{\delta_t = k \sigma_{i,t} \sqrt{\Delta t}.}$$

Y.32. Take-profit

Jika reward-risk ratio:

$$\rho = \frac{TP - \text{entry}}{\text{entry} - SL},$$

maka:

$$TP = \text{entry} + \rho(\text{entry} - SL)$$

untuk long.

Sehingga:

$$TP_t = P_t + \rho \delta_t.$$

Y.33. Expected value trade

Jika:

$$p_w$$

adalah probability of winning,

dan:

$$p_l = 1 - p_w,$$

dengan reward:

$$R$$

dan risk:

$$L,$$

maka:

$$EV = p_w R - p_l L.$$

Trade hanya layak jika:

$$EV > 0.$$

Lebih realistis:

$$EV_{net} = p_w R - p_l L - Cost.$$

Maka:

$$EV_{net} > 0.$$

Y.34. Ini menghubungkan optimizer dengan TP/SL

Sebelumnya portfolio optimizer menghasilkan:

$$x^*.$$

Sekarang kita evaluasi apakah trade individual memenuhi:

$$EV_{net} > 0.$$

Jadi:

$$\text{Portfolio Selection} \boxplus \text{Trade Acceptance}.$$

Keduanya adalah layer berbeda.

Y.35. Order book

Misalkan best bid:

$$B_t$$

dan best ask:

$$A_t.$$

Spread:

$$S_t = A_t - B_t.$$

Mid-price:

$$M_t = \frac{A_t + B_t}{2}.$$

Entry order harus mempertimbangkan:

$$M_t, S_t, Depth_t.$$

Y.36. Probability of fill

Untuk limit order:

$$P(Fill) = p_{fill}(q, d, OB_t, \lambda_t).$$

dengan:

- q : quantity,
- d : distance dari mid,
- OB_t : order book,
- λ_t : order-flow state.

Y.37. Expected value of placing order

Ini menjadi menarik.

Misalkan:

$$p_f = P(Fill).$$

Expected utility:

$$EU = p_f EV_{trade} - (1 - p_f) C_{opportunity}.$$

Jadi kita tidak hanya bertanya:

"Apakah trade profitable?"

Tetapi:

"Apakah menunggu limit order lebih baik daripada langsung market order?"

Y.38. Market order vs limit order

Misalkan:

$$EU_{limit}$$

dan:

$$EU_{market}.$$

Pilih:

$$a_t = \arg \max \{EU_{limit}, EU_{market}, EU_{wait}\}.$$

Sekarang execution juga menjadi optimization problem.

Y.39. Posisi terbuka

Setelah order filled:

$$position_t \in 0.$$

Kita memasuki state baru.

Definisikan:

$$P_t^{pos}$$

sebagai entry price.

Floating PnL:

$$PnL_t = q_t(P_t - P_t^{pos})$$



untuk long.

Y.40. Position state

Kita dapat definisikan:

$$z_t^{pos} = \{entry, SL, TP, PnL, MFE, MAE, age\}.$$

dengan:

- MFE = Maximum Favorable Excursion,
 - MAE = Maximum Adverse Excursion.
-

Y.41. Inilah tempat feedback dimulai

Setiap periode:

$$S_t^{pos}$$

dievaluasi.

Kita hitung:

$$EV_{continue}.$$

Bandingkan dengan:

$$EV_{exit}.$$

Kemudian:

$$a_t^{position} = \arg \max \{EV_{hold}, EV_{reduce}, EV_{exit}, EV_{reverse}\}.$$

Y.42. Floating loss tidak otomatis berarti exit

Ini penting.

Jika:

$$PnL_t < 0,$$

bukan berarti:

$$Exit.$$

Kita harus bertanya:

$$P(Recovery|F_t)$$

dan:

$$EV_{hold}.$$

Jika thesis masih valid:

$$EV_{hold} > EV_{exit},$$

maka posisi dapat dipertahankan.

Y.43. Floating profit juga tidak otomatis berarti TP

Jika:

$$PnL_t > 0,$$

kita mengevaluasi:

$$EV_{hold}$$

versus:

$$EV_{exit}.$$

Jadi keputusan tidak berdasarkan:

"sudah profit, tutup."

Melainkan:

"Apakah expected future reward masih lebih besar daripada expected downside dan opportunity cost?"

Y.44. Dynamic stop-loss

Kita dapat mendefinisikan stop:

$$SL_t = f(entry, PnL, volatility, regime, orderflow).$$

Contohnya:

$$SL_t = \max(SL_{t-1}, P_t - k\sigma_t)$$

untuk long.

Karena:

$$SL_t \geq SL_{t-1},$$

stop tidak boleh mundur.

Y.45. Break-even

Jika profit mencapai threshold:

$$PnL_t \geq \theta_{BE},$$

kita dapat memindahkan:

$$SL_t \rightarrow entry + \epsilon.$$

Secara matematis:

$$SL_t = \max(SL_{t-1}, entry + \epsilon).$$

Y.46. Dynamic TP

Sekarang ide yang sebelumnya kamu berikan dapat diformalkan.

Jika:

$$EV_{hold} > EV_{exit},$$

maka kita dapat meningkatkan target:

$$TP_t \rightarrow TP_{t+1}.$$

Misalnya:

$$TP_{t+1} = P_t + \rho_t \delta_t.$$

dengan:

$$\rho_{t+1} > \rho_t$$

jika momentum/regime masih mendukung.

Y.47. Tetapi jangan menaikkan TP hanya karena profit

Kita membutuhkan condition:

$$\boxed{\frac{\dot{\mu}_t}{\dot{\sigma}_t} > \theta_{continuation}.}$$

Jika:

signal strength

melemah, TP justru dapat diturunkan atau posisi ditutup.

Y.48. State transition

Sekarang kita bisa menuliskan sistem sebagai:

$$s_{t+1} = F(s_t, a_t, \xi_{t+1})$$

dengan:

$$\xi_{t+1}$$

adalah random market shock.

Keputusan:

$$a_t = \pi_\theta(s_t).$$

Jadi keseluruhan strategy adalah policy:

$$\pi : S \rightarrow A.$$

Y.49. Objective global

Sekarang kita sampai pada formulasi yang benar-benar menyatukan semuanya.

Kita tidak ingin hanya memaksimalkan satu-period return.

Kita ingin:

$$\max_{\pi} E \left[\sum_{t=0}^T \gamma^t U(R_{t+1}, Risk_{t+1}, Cost_{t+1}) \right].$$

Misalnya:

$$U_t = R_{t+1} - \lambda_t Risk_t - C_t.$$

Maka:

$$\max_{\pi} E \left[\sum_t \gamma^t (R_{t+1} - \lambda_t Risk_t - C_t) \right].$$

Ini adalah **global control objective**.

Y.50. Tetapi optimizer quantum tidak menyelesaikan objective global secara langsung

Ini distinction penting.

VQE/QAOA kita gunakan untuk menyelesaikan **local combinatorial optimization**:

$$x_t^* = \arg \min_x H_t(x).$$

Sedangkan:

$$\pi$$

adalah keseluruhan trading policy.

Jadi arsitektur kita adalah:

$$\text{Global Policy} \rightarrow \text{Local Optimization} \rightarrow \text{Execution}.$$

Y.51. Local-to-global hierarchy

Secara formal:

$$\pi_\theta : S_t \rightarrow Q_t$$

kemudian:

$$Q_t \rightarrow x_t^*$$

kemudian:

$$x_t^* \rightarrow w_t^*$$

kemudian:

$$w_t^* \rightarrow q_t^*$$

kemudian:

$$q_t^* \rightarrow \text{Order}_t.$$

Y.52. Ising mapping

QUBO:

$$H(x) = x^T Q x + c^T x + c_0.$$

Gunakan:

$$x_i = \frac{1 - z_i}{2}$$

dengan:

$$z_i \in \{-1, +1\}.$$

Substitusi menghasilkan:

$$H(z) = C + \sum_i h_i z_i + \sum_{i < j} J_{ij} z_i z_j.$$

Kemudian:

$$\hat{H} = CI + \sum_i h_i Z_i + \sum_{i < j} J_{ij} Z_i Z_j.$$

Ini adalah Hamiltonian yang masuk ke VQE/QAOA.

Y.53. VQE

VQE mencari:

$$\theta^* = \arg \min_{\theta} \langle \psi(\theta) | \hat{H} | \psi(\theta) \rangle.$$

Kemudian measurement menghasilkan:

$$z^*.$$

Konversi:

$$z^* \rightarrow x^*.$$

Y.54. QAOA

QAOA menggunakan:

$$U(\gamma, \beta) = \prod_p e^{-i\beta_p H_M} e^{-i\gamma_p H_C}.$$

State:

$$|\psi(\gamma, \beta)\rangle = U(\gamma, \beta)|+\rangle^{\otimes N}.$$

Kemudian:

$$\langle H_C \rangle$$

dioptimalkan.

Y.55. Yang menarik: γ, β bukan market variables

Mereka adalah:

variational parameters.

Sedangkan:

$$\mu_t, \Sigma_t$$

adalah market-dependent quantities.

Jadi:

$$\gamma, \beta \notin \mathcal{F}_t$$

dalam arti yang sama seperti market state.

Mereka merupakan parameter yang dipilih untuk menghasilkan solusi dari:

$$H_t.$$

Y.56. Dynamic Hamiltonian

Ini merupakan bagian yang sangat penting dari sistem kita.

Hamiltonian sebenarnya berubah terhadap waktu:

$$\hat{H}_t = \hat{H}(\hat{\mu}_t, \hat{\Sigma}_t, Risk_t, Cost_t, Constraints_t).$$

Jadi kita tidak mempunyai satu Hamiltonian permanen.

Kita memiliki:

$$\hat{H}_1, \hat{H}_2, \dots, \hat{H}_T.$$

Y.57. Dengan demikian trading loop menjadi:

$$\hat{H}_t \rightarrow VQE/QAOA \rightarrow x_t^* \rightarrow Trade_t$$

kemudian market berubah:

$$Market_{t+1} \rightarrow \hat{H}_{t+1}.$$

Kemudian:

$$x_t^*$$

tidak lagi dianggap optimal secara permanen.

Y.58. Ini menjawab masalah "mengapa harus re-optimize?"

Karena:

$$\hat{H}_{t+1} \not\equiv \hat{H}_t.$$

Perubahan dapat berasal dari:

$$\Delta\mu, \Delta\Sigma, \Delta regime, \Delta cost, \Delta liquidity.$$

Jadi:

$$x_t^*$$

dapat menjadi suboptimal.

Y.59. Re-optimization trigger

Kita tidak harus menjalankan optimizer setiap tick.

Definisikan:

$$\Delta H_t = \|H_t - H_{t-1}\|.$$

Re-optimize jika:

$$\Delta H_t > \epsilon_H.$$

Atau jika:

$$|\Delta regime| = 1.$$

Atau:

$$|\Delta \mu_i| > \epsilon_\mu.$$

Atau:

$$\Delta \sigma > \epsilon_\sigma.$$

Y.60. Jadi kita memiliki event-driven architecture

Optimizer aktif ketika:

$$Event_t = \{SignalChange, RegimeChange, RiskChange, PositionChange, LiquidityChange\}.$$

Ini jauh lebih efisien daripada:

Optimize

setiap saat tanpa alasan.

Y.61. Full unified equation

Sekarang seluruh framework dapat diringkas:

$$s_t = [D_{0:t}, w_t, q_t, Risk_t]$$

kemudian:

$$(\hat{\mu}_t, \hat{\Sigma}_t) = \Phi_\theta(s_t)$$

kemudian:

$$H_t(x) = -\hat{\mu}_t^\top x + \lambda_t x^\top \hat{\Sigma}_t x + C_t(x, x_{t-1}) + P_t(x)$$

kemudian:

$$x_t^* = \text{Solve}(H_t)$$

dengan:

$$\text{Solve} \in \{Exact, MILP, Classical, VQE, QAOA\}.$$

Kemudian:

$$w_t^* = \Psi(x_t^*, s_t)$$

dan:

$$q_t^* = \min [q_t^{portfolio}, \frac{R_t}{\delta_t}].$$

Kemudian:

$$O_t = \Omega(w_t^*, q_t^*, OB_t, SL_t, TP_t).$$

Execution menghasilkan:

$$s_{t+1} = F(s_t, O_t, \xi_{t+1}).$$

Y.62. Closed-loop formulation

Akhirnya:

$$\begin{aligned} F_t &\rightarrow s_t \\ &\rightarrow (\mu_t, \Sigma_t) \\ &\rightarrow H_t \\ &\rightarrow x_t^* \\ &\rightarrow w_t^* \\ &\rightarrow q_t^* \\ &\rightarrow O_t \\ &\rightarrow E_{t+1} \\ &\rightarrow R_{t+1} \\ &\rightarrow s_{t+1}. \end{aligned}$$

Dan:

$$s_{t+1}$$

kembali masuk ke sistem.

Itulah feedback loop.

Y.63. Formulasi sebagai constrained stochastic control

Kalau kita ingin menuliskannya dalam bentuk yang lebih formal:

$$\begin{aligned} \max_{\pi} \quad & E^{\pi} \left[\sum_{t=0}^T \gamma^t U(s_t, a_t) \right] \\ \text{s.t.} \quad & s_{t+1} = F(s_t, a_t, \xi_{t+1}), \\ & a_t \in A(s_t), \\ & Risk_t \leq R_t, \\ & Turnover_t \leq \tau_t, \\ & Leverage_t \leq L_t, \\ & Liquidity_t \geq L_t^{min}. \end{aligned}$$

Ini adalah **master formulation**.

Y.64. Tetapi ada dua level optimization

Ini menurut saya adalah desain paling penting dari seluruh framework kita:

Level 1 — Strategic / Policy

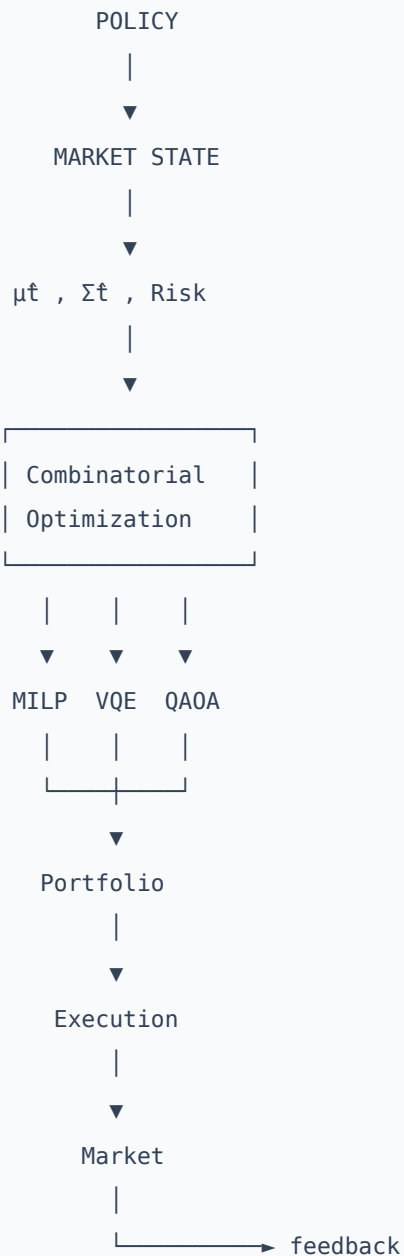
memilih bagaimana sistem mengambil keputusan.

Sedangkan:

Level 2 — Combinatorial Optimization

menentukan portfolio/order candidate pada state tertentu.

Secara diagram:



Y.65. Mengapa desain ini bagus untuk proyek kita?

Karena kita **tidak memaksakan quantum computing untuk menyelesaikan seluruh trading problem**.

Quantum computing hanya digunakan di tempat yang memang natural:

discrete combinatorial optimization.

Sementara:

- factor estimation,
- covariance estimation,

- regime detection,
- risk management,
- execution,

tetap dapat menggunakan classical methods yang memang sesuai.

Ini jauh lebih defensible secara ilmiah daripada mencoba membuat "semua bagian trading menjadi quantum".

Y.66. Dan ini juga membuka banyak jenis problem

Framework yang sama dapat menghasilkan:

Portfolio selection

$$\min_x H(x)$$

MaxCut

$$\max_x \sum_{ij} w_{ij} x_i x_j$$

untuk clustering/exposure structure.

TSP

Untuk routing order execution atau allocation scheduling tertentu:

$$\min_{\pi} \sum_{i,j} d_{ij} x_{ij}.$$

Knapsack

Untuk capital allocation:

$$\max_x \sum_i v_i x_i$$

subject to:

$$\sum_i c_i x_i \leq C.$$

Assignment

Untuk allocation capital/order capacity.

Dengan demikian:

$$QAOA/VQE$$

bukan terikat pada portfolio saja.

Y.67. Dan akhirnya kita bisa melihat visi besar sistem

Sistem kita bukan:

"bot yang membeli crypto."

Melainkan:

Adaptive Multi-Asset Quantitative Decision System

yang mengambil:

market + macro + cross-asset + risk + execution

dan menghasilkan:

discrete trading decisions.

Y.68. Formulasi final yang saya sarankan kita "kunci"

Saya akan menjadikan persamaan berikut sebagai **central equation** dari seluruh proyek:

$$x_t^* = \arg \min_{x \in X_t} [-\hat{\mu}_t^\top x + \lambda_t x^\top \hat{\Sigma}_t x + C_t(x, x_{t-1}) + P_t(x)]$$

dengan:

$$(\hat{\mu}_t, \hat{\Sigma}_t) = \Phi_\theta(P_{0:t}, F_{0:t}, OB_{0:t}, z_t)$$

dan:

$$q_t^* = \min(q_t^{\text{portfolio}}, \frac{R_t}{\delta_t}).$$

Kemudian keputusan dieksekusi melalui:

$$O_t = \Omega(x_t^*, q_t^*, OB_t, SL_t, TP_t).$$

Dan seluruh sistem berevolusi:

$$s_{t+1} = F(s_t, O_t, \xi_{t+1}).$$

Y.69. Satu hal yang perlu kita luruskan

Ada perbedaan antara **model penelitian** dan **bot trading production**.

Untuk penelitian, kita ingin:

$$\text{correctness} > \text{speed}.$$

Kita harus memastikan:

- no leakage,
- valid mathematical formulation,
- correct QUBO,
- correct Ising mapping,
- correct execution simulation,
- robust OOS result.

Baru setelah itu:

$$\text{speed} \rightarrow \text{latency} \rightarrow \text{deployment}.$$

Y.70. Posisi kita sekarang

Dengan Tahap Y, arsitektur kita sudah berubah dari sekadar:

$$A \rightarrow B \rightarrow C \rightarrow \dots$$



menjadi satu persamaan besar:

$$F_t \rightarrow s_t \rightarrow \Phi_\theta \rightarrow (\mu_t, \Sigma_t) \rightarrow H_t \rightarrow \text{Solve} \rightarrow x_t^* \rightarrow w_t \rightarrow q_t \rightarrow \text{Order} \rightarrow \text{Execution} \rightarrow PnL \rightarrow s_{t+1}.$$

Dan menurut saya, **Tahap Y** adalah titik di mana proyek ini akhirnya memiliki "mathematical spine" yang jelas.

Tahap berikutnya yang paling natural adalah **Tahap Z — Concrete Model Specification & Implementation**

Blueprint: kita berhenti pada formulasi abstrak dan menentukan **persis** variabel apa yang digunakan, timeframe, asset universe, feature/factor matrix, estimasi μ_t , estimasi Σ_t , regime model, bentuk QUBO final, encoding qubit, constraint, penalty coefficient, VQE/QAOA circuit, serta format data yang harus masuk dan keluar dari setiap modul. Dengan itu kita bisa mulai menerjemahkan formulasi Y menjadi arsitektur kode yang benar-benar dapat dibangun dan diuji.